



ESTÁNDAR CODIFICACIÓN SEGUNDA VERSIÓN

Principios de Construcción de Software



17 DE MAYO DE 2026
SAMUEL CARRETO BARRIENTOS
DENISSE YAMILETH JUAREZ REYES

Introducción

En este documento se definen los estándares de codificación para nuestro proyecto basándonos en el “Google Java Style Guide” que aseguran una consistencia en el código, logrando que sea fácil de entender, desarrollar y mantener por cualquier integrante del equipo, además de unificar el formato para evitar la pérdida de puntos en evaluaciones y certificaciones. Este estándar no solo cubre cuestiones estéticas, sino también, convenciones que nos permitirá desarrollar un producto más unificado. Se utilizará el idioma inglés para el desarrollo del proyecto con el fin de que sea más comprensible para otros desarrolladores.

Propósito

El objetivo de este documento es establecer una definición de las reglas y convenciones que regirán la escritura de nuestro código fuente. Se busca que el código cumpla con las siguientes características:

- **Mantenibilidad:** Se busca que sea comprensible, escalable y fácil de mantener.
- **Detección temprana de errores:** Al definir claramente cómo debe verse el código “correcto”, logramos que los patrones desconocidos o errores potenciales sean más fáciles de identificar.
- **Productividad:** El seguir estas prácticas incrementa la productividad general al evitar las discusiones innecesarias sobre el formato y centrar el esfuerzo en la resolución de problemas complejos.⁴

Reglas de nombrado

Variables

Los nombres de las variables se definirán usando camelCase, iniciando con minúscula y poniendo en mayúscula la primera letra de cada palabra interna. No se deben utilizar prefijos o sufijos especiales, a excepción de componentes de interfaz y booleanos. Los nombres deberán ser sustantivos o frases que actúen como descriptores completos en inglés, permitiendo entender su propósito de uso. Se deben evitar nombres de un solo carácter, excepto para variables donde el alcance es muy reducido como los contadores de bucles (i, j, k) o excepciones (e) donde el bloque de código no supere las 3 líneas.

Con estándar
<code>int studentCount = 10;</code>
Sin estándar

```
int c = 0;
```

Se usarán nombres en plural para variables que representan colecciones (arreglos, listas o vectores) para indicar que contiene múltiples valores

Con estándar
List<String> customerOrders;
Sin estándar
List<String> customerOrder;

Las variables de tipo booleanos deberán utilizar prefijos como is, has o can (isStudent, hasCustomer).

Con estándar
private boolean isStudent;
Sin estándar
private boolean student;

Se debe declarar una sola variable por la línea de código para mejorar la claridad. Igualmente se debe evitar el sombreado de datos, que consiste en nombrar una variable local igual que una de mayor ámbito, para evitar confusiones de lógica

Con estándar
private String firstName; private String lastName ;
Sin estándar
private String firstName, lastName;

Componentes de interfaz JavaFX

Para componentes de interfaz, se usará el tipo de componente como prefijo(completo), seguido de un descriptor de su funcionalidad

La anotación @FXML deberá colocarse en la misma línea que la declaración del atributo cuando se trate de un componente de interfaz

Con estándar
@FXML private Button buttonLogin;
Sin estándar
@FXML private Button btnLogin;

Identificadores de FXML (fx:id)

El valor del atributo fx:id en el archivo .fxml debe coincidir exactamente con el nombre de la variable declarada en el controlador, el fx:id solo se utilizará en los componentes que sean necesarios modificar o consultar, elementos que no sean utilizados a nivel programación no deben tener un fx:id asignado

Con estándar
<Button fx:id="buttonLogin" text="Iniciar sesión" onAction="#handleLogin"/>
Sin estándar
<Button fx:id="btnLogin" text="Iniciar sesión"/>

Constantes

Los nombres de las constantes se escribirán usando UPPER_SNAKE_CASE en mayúsculas y separadas por un guion bajo.

Las constantes se utilizarán cuando el contenido de las variables sea estático e inmutable y cuyos métodos no tengan efectos secundarios, si el valor de dicha variable cambia no se considera constante.

A continuación, se presentan ejemplos, uno donde se cumple la norma y otro en el que se viola la norma:

Con estándar
<code>static final int MAX_USERS = 50;</code>
Sin estándar
<code>static final int maxusers = 50;</code>

Métodos

Los nombres de los métodos se escribirán usando lowerCamelCase. Sus nombres deberán ser verbos o frases verbales que describen su acción. Esto incluye a los métodos de tipo JUnit para realizar todo tipo de pruebas. Además, los métodos deberán recibir como máximo 3 parámetros, con el fin de mantenerlo claro y legible.

Con estándar
<pre>public void saveUserLog(User user) { // method logic }</pre>
Sin estándar
<pre>public void SaveUserLog(String firstName, String lastName, String email, int userID) { // method logic }</pre>

Manejadores de eventos

Los métodos que actúan como manejadores de eventos en controladores JavaFX deberán comenzar con un verbo de acción (handle, on, toggle) o describir directamente la acción que ejecutan (validateFields, navigateToMenus). Deberán estar anotados con @FXML.

Con estándar

<pre>@FXML public void handleLogin() { // logic }</pre>
Sin estándar
<pre>public void click() { //logic }</pre>

Métodos de prueba unitaria

Los métodos de prueba unitaria deberán seguir el formato nombreMetodo_escenario_resultadoEsperado, separando con guiones bajos las tres partes. Esto permite identificar de un vistazo qué se está probando, en qué condiciones y qué resultado se espera.

Con estándar
<pre>@Test void registerUser_succesful_returnsGeneratedId() throws Exception { //logic</pre>
<pre>}</pre>
Sin estándar
<pre>@Test void test1() { //logic }</pre>

Clases

El nombre de las clases se definirá usando UpperCamelCase(también conocido como PascalCase). Los nombres de las clases suelen ser sustantivos o frases nominales. Se usará el sufijo DAO para las clases de acceso de datos con el fin de mostrar una diferencia entre las clases de la capa de datos y los POJO(Objeto Java Simple).

Con estándar

```
public class UserDao implements IUserDAO {

    // method DAO
}
```

Sin estándar

```
public class daouser {

    // method dao
}
```

Clases controladoras

Las clases que actúan como controladores de una vista JavaFX deberán comenzar con el prefijo FXML, seguido del nombre de la vista que controlan y terminar con el sufijo Controller. El archivo .fxml asociado debe llevar el mismo nombre que el controlador, sin el sufijo Controller.

Con estándar

```
public class FXMLLoginController implements Initializable {
    //implementation
}
```

Sin estándar

```
public class LoginCtrl implements initializable {
    //implementation
}
```

Interfaces

Las interfaces que definen los métodos de las clases DAO deberán llevar el prefijo "I" seguido del nombre de la clase concreta

Con estándar

```
public interface IStudentDAO {  
    //implementation  
}
```

Sin estándar

```
public interface InterfaceStudent {  
    //implementation  
}
```

Sufijos por responsabilidad

Las clases deberán llevar sufijos consistentes según el rol que cumplen dentro del sistema:

- DAO: clases de acceso a datos (UserDAO, ProfessorDAO).
- Controller: controladores de vistas JavaFX (FXMLLoginController).
- Service: clases de servicio en la capa lógica (AutoevaluationService).
- Handler: clases base que contienen comportamiento de vistas (ValidationHandler).
- Validator: clases de validación (InputValidator, FileValidator).
- Common: lógica auxiliar reutilizable (EmailCommon)
- Cell: celdas personalizadas de TableView o ListView (StudentListView)
- Exception: clases de excepciones personalizadas (OperationException).

Organización del proyecto

Para la elaboración del proyecto se utilizará el lenguaje de programación Java, empleando Visual Studio Code como entorno de desarrollo. Como se mencionó en la introducción, se establecerá el inglés como idioma principal para la escritura de todo el código fuente. A nivel estructural, el sistema se construirá bajo un modelo de arquitectura de tres capas: la capa lógica, la capa de acceso a datos y la interfaz gráfica de usuario (GUI). La implementación de este patrón tiene como propósito fundamental garantizar la estricta separación de responsabilidades, aislando la interacción visual de las reglas del negocio y del almacenamiento de la información.

Capa acceso de datos

Para la persistencia y gestión de la información, se utilizará MySQL como sistema gestor de bases de datos relacionales. A diferencia del código fuente, el diseño de la

base de datos y el almacenamiento de los registros se realizarán en español; esto se justifica con el objetivo de extraer y presentar la información directamente en el idioma que requiere el usuario final, optimizando la claridad de la interfaz gráfica y la generación de reportes, se empleará el entorno de desarrollo MySQL Workbench para su diseño y administración a través del lenguaje de consulta estructurado SQL.

Dentro de esta arquitectura, la capa de acceso a datos jugará un papel fundamental en la comunicación, ya que será el único componente del sistema diseñado y autorizado para conectarse directamente y manipular la base de datos. Dentro de esta capa se definirán las clases específicas que se conectarán con la base de datos mediante la implementación del patrón de diseño DAO (*Data Access Object*). Concentrar todas las operaciones de extracción y modificación de registros exclusivamente en esta capa asegura que el resto del programa no interactúe con el motor de base de datos de forma desordenada.

Contratos e implementaciones DAO

Cada clase DAO deberá implementar una interfaz correspondiente. Esto facilita la inyección de dependencias en las pruebas unitarias mediante mocks y permite cambiar la implementación concreta sin afectar al resto del sistema. Los métodos declarados en la interfaz deben coincidir exactamente en firma con los métodos públicos de la clase DAO.

Con estándar
<pre>public class UserDao implements IUserDAO { @Override public int registerUser(User user) throws OperationException{ //logic } }</pre>
Sin estándar
<pre>public class UserDao { //implementation }</pre>

Capa Lógica

En esta capa estará la lógica y el funcionamiento del programa. No se encargará de mostrar interfaces ni de conectarse directamente a la base de datos. Su función principal será procesar la información recibida desde la interfaz gráfica, aplicar las reglas de negocio del sistema y coordinar las operaciones necesarias para que el

programa funcione correctamente.

Esta capa actuará como intermediaria entre la interfaz de usuario y la capa de acceso a datos, asegurando que los datos sean validados y procesados antes de ser almacenados o consultados en la base de datos

Capa GUI

En esta capa se encontrará la interfaz gráfica del sistema, la cual será la encargada de permitir la interacción entre el usuario y el programa. Su función principal es mostrar la información al usuario y capturar los datos que este ingrese, como formularios, botones, campos de texto y mensajes del sistema.

Esta capa no contendrá conexiones directas a la base de datos. En su lugar, la capa lógica será la responsable de procesar y determinar las acciones que deben realizarse.

Los textos y mensajes mostrados en la interfaz gráfica serán presentados en español, con el objetivo de que coincidan con el idioma del usuario final y faciliten la comprensión del sistema durante su uso.

Separación FXML / Controller

Cada vista del sistema deberá estar compuesta por dos archivos: un archivo .fxml que define la estructura visual y un archivo .java que implementa la lógica del controlador. No se deben construir interfaces gráficas mediante código Java puro cuando exista una alternativa con FXML, salvo componentes dinámicos como ImageView, tablas con celdas personalizadas, etc...

Control de versiones

El proyecto utilizará Git como sistema de control de versiones y GitHub como repositorio remoto para el almacenamiento y gestión del código fuente.

Todo cambio realizado en el proyecto deberá registrarse mediante commits. Cada commit debe incluir un mensaje descriptivo que explique claramente el cambio realizado en el código. Esto permite llevar un historial claro de las modificaciones y facilita el seguimiento del desarrollo del proyecto.

Los mensajes de commit deberán describir de manera breve y clara la modificación realizada, evitando mensajes genéricos o poco informativos, para facilitar la comprensión, estos mensajes, a diferencia del código, estarán en español.

Con estándar
Se arreglo el error de conexión a la base de datos

Sin estándar
Arreglado

Estilo de código

Formato y Estructura General

Indentación: Por cada nivel de Indentación se utilizarán 4 espacios, el uso de tabulador debe ser el mínimo posible para este propósito

Con estándar
<pre>if (condition) { if(anotherCondition) { //logic } }</pre>
Sin estándar
<pre>if (condition) { if(anotherCondition) { //logic } }</pre>

Límite de columna: El código debe mantenerse dentro de un límite de 120 caracteres por línea, cuando una instrucción sea demasiado larga, se deberá romper después de una coma o antes de un operador. Las líneas de continuación deben indentarse al menos 4 espacios adicionales respecto a la línea original.

Con estándar
<pre>User user = new User (firstName, lastName, email, userId, entryTime, exitTime, entryDate, exitDate);</pre>
Sin estándar

```
User user = new User (firstName, lastName, email, userId, entryTime, exitTime,
entryDate, exitDate);
```

Uso de espacios en Blanco

- Vertical
 - Se debe dejar una línea en blanco entre métodos, constructores y después de la última variable de instancia.
 - Se usará una línea en blanco para separar las secciones del archivo: una línea entre la declaración del paquete y el bloque de importaciones y una línea entre el bloque de importaciones y la declaración de la clase.
 - Dentro del bloque de importaciones, se usará una línea en blanco entre los subgrupos: importaciones estáticas, importaciones de la biblioteca estándar de Java, importaciones de javaFX(si aplican) e importaciones del propio proyecto,

Con estándar

```
package com.example.app

import java.Util.ArrayList;

public class Visitor {
    private String firstName;
    private String lastName;

    public Visitor(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }

    public String getFirstName() {
        return firstName;
    }
}
```

Sin estándar

```

package com.example.app
import java.Util.ArrayList;
public Class Visitor {
    private String firstName;
    private String lastName;
    public Visitor(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }
    public String getFirstName() {

return firstName;
    }
}

```

- Horizontal.

- Las estructuras de control (condicionales, bucles y manejo de excepciones) deben tener un espacio entre la palabra clave y el paréntesis de apertura.
- Colocar espacios alrededor de todos los operadores binarios (==, &&, +) con excepción del punto.
- No debe haber espacio entre el nombre de un método y su paréntesis de apertura.
- Operadores como ++, no deberá llevar espacio.
- Debe colocarse un espacio entre el paréntesis de cierre de la condición y la llave de apertura { en estructuras de control.

Con estándar
<pre> static final int MINIMUM_AGE = 18; static final int MAXIMUM_AGE = 65; if (userAge >= MINIMUM_AGE && userName != " " && isActive == true) { while (userAge < MAXIMUM_AGE) { userAge++; } } </pre>
Sin estándar

```
if(userAge>=18&&userName!=" "&&isActive==true){  
  while(userAge<65){  
    userAge ++;  
  }  
}
```

Organización del Código en el Archivo

El archivo debe seguir este orden:

1. Declaración del paquete.
2. Importaciones (no está permitido el uso de asteriscos *).
 - a. Estáticas
 - b. No estáticas
3. Declaración de la clase.

Con estándar

```
package com.example.app;  
  
import static org.junit.Assert.assertEquals;  
import static java.lang.Math.PI  
import java.util.ArrayList;  
import java.util.HashMap;  
import react.Component;  
  
public class UserManager extends Component {  
  
}
```

Sin estándar

```
import react.Component;  
package com.example.app;  
  
import java.util.*;  
import java.util.ArrayList;  
import static org.junit.Assert.aserEquals;  
  
package com.example.app;  
import react.Component;  
public class UserManager{ }
```

Orden de Miembros en la Clase:

1. Variables constantes.
2. Variables de instancia
3. Constructores.
4. Métodos, ordenados por visibilidad, públicos, protegidos y finalmente privados.

Con estándar

```
public class Visitor() {  
    private static MAX_USERS = 100;  
    private String userName;  
  
    public Visitor(String name) {  
  
    }  
  
    public void getName() {  
  
    }  
}
```

Sin estándar

```
Public class Visitor(String name) {  
    private static maxUsers = 100;  
    private String userName;  
  
    public void getName() {  
  
    }  
  
    public Visitor(String name) {  
  
    }  
}
```

Prácticas de Programación

La anotación “@Override” deberá utilizarse siempre que un método sobrescriba una definición de una superclase o interfaz.

En las excepciones no se permiten bloques catch vacíos. Si llegara a ser absolutamente necesario ignorar una excepción, se deberá justificar con un comentario.

Con estándar

```
@Override
public String getName() {
    return 'Employee: ${super.getName()}';
}

public void loadData() {
    try {
        fetchUserData();
    } catch (error) {
        LOGGER.log(Level.SEVERE, "Error de conexión con la base de datos", e);
        showErrorMessage();
    }
}
```

Sin estándar

```
getName() {
    return 'Employee: ${super.getName()}';

}

loadData() {
    try {
        fetchUserData();
    } catch (error) {
    }
}
```

Se prohíbe el uso de números mágicos dentro del código. Un número mágico es un valor numérico utilizado directamente en una expresión sin una explicación clara de su propósito. En su lugar, se deben definir constantes descriptivas utilizando la palabra clave static final.

Con estándar

```

static final int MINIMUM_AGE = 18;
static final int MAXIMUM_AGE = 65;

if (userAge >= MINIMUM_AGE && userName != " " && isActive == true)
{

    while (userAge < MAXIMUM_AGE) {
    userAge++;
    }
}

```

Sin estándar

```

if (userAge >= 18 && userName != " " && isActive == true) {
    while (userAge < 65) {
    userAge++;
    }
}

```

Número máximo de operadores en expresiones

Para mantener la claridad y legibilidad del código, las expresiones lógicas o condicionales no deberán contener más de tres operadores lógicos o de comparación en una misma condición.

Con estándar

```

boolean isValidAccess = isActive && hasPermission && userAge >=
MINIMUM_AGE && isVerified;

if (isValidAccess) {
    grantAccess();
}

```

Sin estándar

```

if (isActive && hasPermission && userAge >= MINIMUM_AGE && isVerified)
{
    grantAccess();
}

```

Complejidad ciclomática

El máximo de complejidad ciclomática que puede contener un método será de 12, para evitar métodos muy complejos o difíciles de mantener, siempre se buscará mantener la complejidad lo más baja posible.

Si es necesario, se deberá refactorizar o separar el método con el propósito de hacerlo más simple

Con estándar

```
public double processOrder(Order order, User user, Map<String, Integer>
inventory) {
    double result = -1;

    if (isOrderRequestValid(order, user)) {
        double total = calculateOrderTotal(order.getItems(), user, inventory); boolean
orderCanBeProcessed = total >= 0 && isPaymentAllowed(total, user);

        if (orderCanBeProcessed) {
            double shipping = calculateShippingCost(order, total);
            result = total + shipping;
        }
    }

    return result;
}

private boolean isOrderRequestValid(Order order, User user) {
    //logic
}

private boolean isPaymentAllowed(double total, User user) {
    //logic
}

private double calculateOrderTotal(List<Item> items, User user, Map<String,
Integer> inventory) {
    //logic
}

private double calculateItemTotal(Item item, User user, Map<String, Integer>
```

```
inventory) {  
    //logic;  
}  
  
private int resolveAvailableQuantity(Item item, User user, Map<String, Integer>  
inventory) {  
    //logic  
}  
  
private double resolveUnitPrice(Item item, User user) {  
    //logic  
}  
  
private double calculateShippingCost(Order order, double total)  
{  
  
    //logic  
}
```

Sin estándar

```
public double processOrder(Order order, User user, Map<String, Integer>
inventory) {
    double result = -1;

    if (order != null) {
        if (user != null && user.isActive()) {
            double total = 0;
            boolean valid = true;

            for (Item item : order.getItems()) {
                if (item.getQuantity() > 0) {
                    if (inventory.containsKey(item.getId())) {
                        int stock = inventory.get(item.getId());

                        if (stock >= item.getQty()) {
                            double price = item.getPrice();

                            if (user.hasDiscount() && item.isDiscountable()) {

                                price *= 0.9;
                            }

                            total += price * item.getQty();
                        } else {
                            if (user.isVip()) {
                                total += item.getPrice() * stock;
                            } else {
                                valid = false;
                            }
                        }
                    } else {
                        valid = false;
                    }
                }
            }
        }
    }
}
```

```

    }
    }
    }

    if (valid && (total <= 1000 || user.isCreditApproved())) {
    if (order.isExpress() || total < 50) {
    total += 15;
    }
    result = total;
    }
    }
    }

    return result;
}

```

Uso de Optional

Cuando un método pueda devolver un resultado válido o ningún resultado (por ejemplo, una búsqueda en base de datos que no encuentra registros), se deberá utilizar `java.util.Optional<T>` en lugar de devolver `null`. Esto maneja explícitamente el caso vacío y elimina los `NullPointerException`.

Reglas para el uso de Optional:

- Nunca devolver `null` desde un método que declare `Optional` como tipo de retorno.
- Para encadenar validaciones que devuelven `Optional<String>`, usar `or()` o [Stream.of\(...\).filter\(Optional::isPresent\).findFirst\(\)](#).

Con estándar

```

public Optional<User> authenticate(String email, String
password) throws AuthenticationException {
    Optional<User> validateUser = Optional.empty();
    //logic
    if(resultSet.next() {
    validateUser = Optional.of(user);
    }
    return validateUser;
}

```

Sin estándar

```

public User authenticate(String email, String password) {
    //logic
    if(!resultSet.next()) {
        return null;
    }
    return user;
}

```

Propagación de Excepciones

- Se utilizará una jerarquía de excepciones personalizadas que extienden de java.lang.Exception para representar los distintos modos de falla del dominio:
 - OperationException: Errores genéricos al ejecutar una operación de negocio o de base de datos(registro, modificación, consulta).
 - AuthenticateException: Errores específicos del proceso de autenticación de usuarios.
 - EmailException: errores específicos del envío de correo electrónico.
- Las SQLException no deben propagarse hacia capas superiores. Deben capturarse en la capa DAO, registrarse con LOGGER.log y empaquetarse en una OperationException preservando la causa original como segundo argumento.
- Los métodos que pueden fallar deben declarar las excepciones en la cláusula throws de su firma, no se debe envolver toda la lógica en un catch genérico de Exception o Throwable.
- El nombre de la variable de la excepción en los bloques catch debe ser descriptivo a menos que el bloque tenga 3 líneas o menos, en cuyo caso se permite la letra e.
- No se debe usar el printStackTrace

Con estándar

```

public int registerUser(User user) throws OperationException {
    try (Connection databaseConnection = connectionManager.getConnection();
        PreparedStatement preparedStatement
        = databaseConnection.prepareStatement(userQuery,
        PreparedStatement.RETURN_GENERATED_KEYS)) {
        //logic
    } catch (SQLException e) {
        LOGGER.log(Level.SEVERE, "Error de conexión con la base de datos", e)
        throw new OperationException("Error al registrar al usuario. Intente más "
        + "tarde", e)
    }
}

```

Sin estándar

```
public int registerUser(User user) throws OperationException {
    try (Connection databaseConnection = connectionManager.getConnection());
    PreparedStatement preparedStatement
    = databaseConnection.prepareStatement(userQuery,
    PreparedStatement.RETURN_GENERATED_KEYS)) {
        //logic

    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

Uso de lambdas

Se permiten las expresiones lambda en los siguientes contextos:

- Listeners de propiedades JavaFX (addListener).
- Manejadores de eventos puntuales sobre componentes creados dinámicamente (MenuItem, TableView, etc.).
- Métodos funcionales de Optional: ifPresent, or, map, filter.
- Métodos de colecciones: forEach, removeIf.
- Operaciones intermedias y terminales de Stream.
- Asignación de Runnable, Supplier o Consumer cuando un método los reciba como parámetro (por ejemplo, handleValidation(error, this::registerSubject)).

Para componentes JavaFX definidos en FXML (botones de pantalla, campos), se debe seguir usando el atributo `onAction="#handleConsult"` con su método anotado con `@FXML`.

Con estándar

```
private void configureTableColumns() {
    columnName.setCellValueFactory(cellData
        -> new SimpleStringProperty(cellData.getValue().getName()));
    columnDocumentType.setCellValueFactory(cellData
        -> new SimpleStringProperty(cellData.getValue().getTypeDocument()));
    columnValidated.setCellFactory(column
        -> new ValidationTableCell(this::handleValidationToggle));
    columnAction.setCellFactory(column
        -> new ActionTableCell(this::openDocument));
}
```

Sin estándar

```
buttonSave.setOnAction(e -> {
    Optional.ofNullable(tableStudents.getSelectionModel().getSelectedItem())
        .filter(student -> student.getGrade() >= 6.0)
        .map(student -> {
            studentDAO.update(student);
            return student;
        })
        .ifPresent(student -> {
            alerts.showSuccess();
            tableStudents.getItems().removeIf(s -> s.getId() == student.getId());
        });
});
```

Uso de Stream

Stream se usará para:

1. Encadenamiento de validaciones que devuelven `Optional<String>`, para obtener el primer error encontrado.
2. Transformación de una colección de DTOs en una colección de uno de sus campos (típicamente para alimentar un `ComboBox` o un `ListView`).

Para iteraciones simples con efectos secundarios (modificar un componente JavaFX, agregar a una colección mutable, registrar logs) se debe seguir usando un bucle `for-each` tradicional.

Con estándar

```
private void loadAvailableProjects() {
    try {
        List<Project> projects = projectDAO.getAllProjectsWithCapacity();
        List<String> projectNames = projects.stream()
            .map(Project::getName)
            .collect(Collectors.toList());

        comboBoxProjects.setItems(FXCollections.observableArrayList(projectNames));
    } catch (OperationException e) {
        showError(e.getMessage());
    }
}
```

Sin estándar

```

private void loadAvailableProjects() {
    try {
        List<Project> projects =
        projectDAO.getAllProjectsWithCapacity();(comboBox)
        projects.stream()
            .forEach(project -> {
                comboBoxProjects.getItems().add(project.getName());
            });
    } catch (OperationExcepcion e) {
        showError(e.getMessage());
    }
}

```

Uso de salida estándar

No se permite el uso de `System.out.println()` ni `System.err.println()` para mostrar mensajes dentro del sistema. Estas instrucciones sólo podrán utilizarse en pruebas temporales durante el desarrollo y deberán eliminarse antes de integrar el código al repositorio principal. Tampoco se permite `printStackTrace()`, ya que escribe directamente a la salida de error sin pasar por el sistema de *logging* ni respetar los niveles configurados.

Para registrar mensajes del sistema, errores o información de ejecución se utilizará el sistema de *logging* incluido en la biblioteca estándar de Java: `java.util.logging.Logger`. A diferencia de la salida estándar, permite clasificar los mensajes por severidad, habilitarlos o silenciarlos por nivel sin tocar el código, y adjuntar la traza completa de una excepción para diagnóstico.

Declaración del *logger*

Cada clase que registre mensajes declarará un único *logger* como constante de clase, nombrado en `UPPER_SNAKE_CASE` e inicializado con el nombre de la propia clase. Esto asegura que cada registro quede etiquetado con su origen.

Niveles de log y cuándo aplicarlos

- **Level.SEVERE:** errores que comprometen una operación crítica (fallo de conexión a la base de datos, excepción no recuperable, recurso indispensable ausente). Cuando exista una excepción asociada, se pasará siempre como último argumento para conservar su traza. Si el fallo crítico no proviene de una excepción (por ejemplo, una plantilla de reporte que no se encuentra), se registrará con un mensaje descriptivo usando marcadores de posición, sin tercer argumento de tipo excepción.
- **Level.WARNING:** situaciones inesperadas pero recuperables (credenciales incorrectas, rol desconocido, consultas sin resultados, fallos de autocompletado,

operaciones no soportadas). Puede registrarse con la excepción asociada o con un mensaje descriptivo, según convenga al diagnóstico.

- Level.INFO: eventos relevantes del flujo normal (inicio de sesión exitoso, registro o actualización completados). Es habitual incluir el identificador de la entidad afectada mediante un marcador de posición.

Marcadores de posición y el tercer argumento

Para incluir valores dinámicos en el mensaje se utilizará la sintaxis con marcadores de posición {0}, {1}, etc., en lugar de concatenar cadenas con +. Esto evita construir la cadena cuando el nivel está deshabilitado y previene errores de formato. La concatenación con + queda prohibida únicamente dentro de las llamadas a `LOGGER.log`

Los tres usos del último argumento de `Logger.log`, no son intercambiables y no pueden combinarse en una sola llamada simple:

- `LOGGER.log(Level, mensaje, excepción)` — adjunta la traza completa de la excepción. El mensaje no contiene marcadores {0}.
- `LOGGER.log(Level, mensaje, valor)` — rellena {0} con un único valor dinámico.
- `LOGGER.log(Level, mensaje, new Object[] { valor0, valor1 })` — rellena varios marcadores {0}, {1}.

Por lo tanto, cuando se necesite registrar tanto un valor dinámico como la causa de un error, se elegirá una de dos estrategias: usar un mensaje estático que describa la operación y pasar la excepción para conservar la traza (el contexto del valor queda en el propio mensaje), o, si basta con el texto del error, insertarlo en línea con {0} mediante `e.getMessage()`.

Los mensajes técnicos para el `LOGGER` se escribirán en español para mantener la consistencia con el resto del equipo.

Con estándar

```
public class ProjectDAO implements IProjectDAO {

    private          static          final          Logger          LOGGER          =
    Logger.getLogger(ProjectDAO.class.getName());

    @Override
    public boolean modifyProject(Project project) throws OperationException {
        boolean isModified = false;
        String projectQuery = "UPDATE Proyecto SET nombre = ? WHERE
idProyecto = ?;";

        try (Connection databaseConnection = connectionManager.getConnection();
```

```

        PreparedStatement preparedStatement =
            databaseConnection.prepareStatement(projectQuery)) {

    //logic

    if (preparedStatement.executeUpdate() > NO_ROWS_AFFECTED) {
        isModified = true;
        LOGGER.log(Level.INFO,
            "Proyecto con ID {0} modificado con éxito.", project.getId());
    } else {
        LOGGER.log(Level.WARNING,
            "No se pudo modificar el proyecto con ID {0}.", project.getId());
    }
} catch (SQLException e) {
    LOGGER.log(Level.SEVERE, "Error de conexión con la base de datos", e);
    throw new OperationException(
        "No se pudo modificar el proyecto. Inténtelo más tarde", e);
}
return isModified;
}
}

```

Sin estándar

```

public class ProjectDAO {

    private Logger logger = Logger.getLogger("ProjectDAO");

    public boolean modifyProject(Project project) {
        boolean isModified = false;
        String query = "UPDATE Proyecto SET nombre = ? WHERE idProyecto = ?";

        try {
            //logic

            if (preparedStatement.executeUpdate() > 0) {
                isModified = true;
                logger.log(Level.INFO, "Proyecto con ID " + project.getId() + " modificado");
            } else {
                System.out.println("No se pudo modificar el proyecto");
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return isModified;
    }
}

```

Comentarios

Comentarios de implementación:

Se utilizarán comentarios de implementación para documentar el código fuente. El código reconoce cuatro estilos de comentarios de implementación: en bloque, de una sola línea, finales y de documentación. Todo fragmento de código que se desvíe de una norma del estándar, que implemente una decisión de diseño no evidente o que resuelva un caso especial deberá estar acompañado de un comentario que explique claramente el motivo de dicha decisión, no únicamente lo que hace el código, sino por qué se optó por esa solución en ese contexto.

Comentarios en bloque:

Estos comentarios se implementarán para proporcionar una justificación de métodos y algoritmos. Pueden usarse antes de los métodos o dentro de ellos y deben ser indicados al mismo nivel del código escrito. Debe ir precedido de una línea en blanco para separarlo del resto del código y se delimitará por `/*...*/`.

Con estándar

```
public class UserManager {  
  
    /* This method was written requiring a username and password for the purpose of  
    maintaining user security */  
  
    public void registerUser() {  
  
    }  
}
```

Sin estándar

```
/******  
 * DATA SECTION *  
******/  
public boolean registerUser() {  
  
}
```

Comentarios de una sola línea y finales:

Se podrán usar comentarios cortos de una sola línea, indentados al nivel del código. Se precederán por una línea en blanco siempre y cuando el comentario aporte claridad, estarán delimitados por `//`.

Se aplicará el comentario en la misma línea que el código, pero deben estar

desplazados lo suficiente para separarlos de las sentencias, se pondrá un comentario para explicar el uso de una variable o cualquier instrucción y también se delimitaran por //.

Con estándar

<pre>public void checkStatus() { initializeHardware(); // we made this conditional revents streaming if initializeHardware() fails. if (isConnected) { startStreaming(); } }</pre>
--

Sin estándar

<pre>public void checkStatus() { initializeHardware(); //verificar si la conexión es estable antes del procedimiento if(isConnected) { } }</pre>
--

Comentario de documentación:

Se establecerán comentarios de documentación los cuales se delimitan por `/**...*/`. Se deben escribir a modo de describir la especificación del código, desde una perspectiva ajena de implementación con el fin de ser leídos por desarrolladores que pueden no tener necesariamente el código fuente. Estos pueden ser extraídos para convertirlos a archivos HTML usando la herramienta Javadoc.

Los comentarios de documentación simples deben contribuir a la claridad del código y deben ser seguidos de una línea en blanco.

Con estándar

```

/**
 * Registers a new user by appending their credentials to the storage file.
 * * @param username The unique identifier for the new user. * @param
password The secret key associated with the account. * @return true if
the registration was successful, false otherwise. */

```

```

public class UserManager {

}

```

Sin estándar

```

public class UserManager {
/* registro de usuario aquí */ public boolean registerUser(String username, String
password) {
/* Technical Justification: Using BufferedWriter with append mode */ try
(BufferedWriter bw = new BufferedWriter(new FileWriter("users.db", true))) {
bw.write(username + "," + password);
bw.newLine();
return true;
} catch (IOException e) {
System.err.println("Error: " + e.getMessage()); return false; }
}
}

```

Comentarios especiales (TODO, FIXME)

Se permitirá el uso de comentarios especiales para indicar tareas pendientes, mejoras o problemas que deben ser corregidos en el código. Estos comentarios ayudan a los desarrolladores a identificar partes del código que requieren atención en futuras modificaciones.

Los comentarios especiales deberán escribirse en mayúsculas y colocarse antes de la sección de código correspondiente, además los comentarios deberán ser claros y descriptivos.

TODO

Se utiliza para indicar funcionalidades o mejoras que aún deben implementarse.

Con estándar

```

//TODO implement email validation
validateEmail(userEmail);

```

Sin estándar
//todo validateEmail(userEmail);

FIXME

Se utiliza para señalar errores o comportamientos incorrectos que deben ser corregidos.

Con estándar
//FIXME this query may generate duplicate records insertVisitRecord(visit);
Sin estándar
//fixme it does not work insertVisitRecord(visit);

Estos comentarios deberán revisarse y resolverse antes de la entrega final del proyecto, con el fin de evitar que código incompleto o errores pendientes permanezcan en el sistema.

Estructuras de control

Llaves

Las llaves, {}, son obligatorias para todas las estructuras de control, incluso si contienen una sola línea de código.

La llave de apertura debe colocarse al final de la línea que inicia la estructura, y la llave de cierre en una línea nueva, alineada con el inicio de la declaración.

Con estándar
if (isActive) { processUser(); }

Sin estándar
<pre>if (isActive) processUser();</pre>

Sentencias Condicionales (if, else)

La palabra clave if y la expresión condicional deben ir en la misma línea. Si existe un else o else if, la llave de cierre del bloque anterior y la palabra clave deben compartir la misma línea (} else {).

Con estándar
<pre>static final int MINIMUM_AGE = 18; if (userAge >= MINIMUM_AGE) { grantAccess(); } else { denyAccess(); }</pre>

Sin estándar
<pre>if (userAge >= 18) { grantAccess(); } else { denyAccess(); }</pre>

Bucles (for, while, do-while)

Se evitará usar más de tres variables en la cláusula de inicialización o actualización de un bucle.

Con estándar
<pre>for (let i = 0; i < arrayLength; i++) { if (user[i].age < maxValue) { processUser(users[i]); } }</pre>

Sin estándar
<pre>for (let i = 0, j = 0, k = 0, count = 0, total = 0; i < 10; i++) { total = i + j + k + count; }</pre>

Sentencias Switch

Todo switch debe incluir una etiqueta default, incluso si no contiene código. Si un bloque de caso no termina en una ruptura (break, return, throw) se debe incluir un comentario explícito como //fall through para indicar que la continuación al siguiente caso es intencional

Con estándar
<pre>switch (userRole) { case 'admin': grantFullAccess(); break; case 'user': grantLimitedAccess(); break; default: denyAccess(); break; }</pre>
Sin estándar
<pre>switch (userRole) { case 'admin'; grantFullAccess(); case 'user'; grantLimitedAccess(); break; }</pre>

Uso del operador ternario

El operador ternario (condición ? valorSiVerdadero : valorSiFalso) podrá utilizarse

únicamente cuando la expresión sea simple y fácil de entender. Su uso debe limitarse a asignaciones sencillas y a establecer texto en componentes JavaFX (setText, setStyle); no debe reemplazar estructuras de control complejas.

- La condición debe ser una sola expresión booleana simple (sin más de un operador lógico).
- Ambas ramas (valor verdadero y valor falso) deben ser expresiones que retornen un valor; no se permite invocar métodos void en sus ramas.
- Las dos ramas deben tener el mismo tipo de retorno y no superar los 80 caracteres en total. Si lo superan, se debe usar un if-else.
- Se prohíbe el anidamiento de operadores ternarios (a ? b : c ? d : e).

Con estándar
String statusMessage = isActive ? "Active user" : "Inactive user";
Sin estándar
String accessResult = (isActive && hasPermission && userAge > 18) ? grantAccess(user) : denyAccess(user);

Conexión con la base de datos

La conexión con la base de datos deberá realizarse únicamente desde la capa de lógica con las clases de acceso a datos (DAO). Ninguna otra capa del sistema está autorizada a interactuar directamente con el motor de base de datos.

Todas las operaciones relacionadas con consultas, inserciones, actualizaciones o eliminación de registros deberán implementarse dentro de las clases DAO, con el objetivo de mantener una correcta separación de responsabilidades dentro del sistema.

Las consultas que se realicen serán guardadas en un String que se llamará empezando con el nombre de la clase en la que está + el sufijo query

Con estándar

```
@Override
public boolean modifyProfessor(Professor professor) throws OperationException
{ boolean isModified = false;

    String professorQuery = "UPDATE Profesor p INNER JOIN Usuario u ON
p.idUsuario = u.idUsuario "
+ "SET p.idRol = ?, u.nombre = ?, u.apellidos = ? "
+ "WHERE p.numeroPersonal = ?";
```

Sin estándar

```
@Override
public boolean modifyProfessor(Professor professor) throws OperationException
{ boolean isModified = false;

    String query = "UPDATE Profesor p INNER JOIN Usuario u ON p.idUsuario =
u.idUsuario "
+ "SET p.idRol = ?, u.nombre = ?, u.apellidos = ? "
+ "WHERE p.numeroPersonal = ?";
```

Usuarios de base de datos

Por motivos de seguridad, no se permitirá utilizar el usuario root para la conexión del sistema con la base de datos. En su lugar, se deberá crear un usuario específico para la aplicación con permisos limitados.

Los permisos permitidos para este usuario serán únicamente los necesarios para el funcionamiento del sistema.

Permisos:

- SELECT
- INSERT
- UPDATE

Esto reduce el riesgo de que un error en la aplicación comprometa la estructura completa de la base de datos.

Uso de Mayúsculas para Palabras Reservadas:

- Todas las palabras clave propias del lenguaje SQL (tales como SELECT,

FROM, INNER JOIN, ON, WHERE, AND, etc.) deben escribirse siempre en MAYÚSCULAS.

- Los nombres de las tablas y los atributos (columnas) deben escribirse respetando la capitalización exacta de la base de datos (por ejemplo, *came/Case* como *numeroPersonal* o *idUsuario*).

Concatenación, Multilínea y Alineación Visual:

- Para evitar líneas de código excesivamente largas, las consultas deben dividirse en múltiples líneas utilizando la concatenación de cadenas de texto (+).
- Alineación a la misma altura: Al dividir la consulta, el inicio de cada nueva línea de texto (las comillas dobles " y el operador +) debe estar alineado verticalmente a la misma altura que la línea superior. Esto crea un bloque de código cuadrado y uniforme.

Credenciales y archivo de configuración

El acceso a la base de datos(URL, usuario, contraseña) no deben estar escritas directamente en el código fuente. Estas residen en un archivo de propiedades externo que será leído por la clase de gestión de conexión.

El archivo debe estar en el .gitignore para no subirse al repositorio.

Con estándar
<pre>database.url=jdbc:mysql://localhost:3306/practicas database.user=appUser database.password=***** public Connection getConnection() throws SQLException { String url = properties.getProperty("database.url"); String user = properties.getProperty("database.user"); String password = properties.getProperty("database.password"); return DriverManager.getConnection(url, user, password); }</pre>
Sin estándar
<pre>return DriverManager.getConnection("jdbc:mysql://localhost:3306/practicas", "root", "123456");</pre>

Manejo de conexiones

Las conexiones con la base de datos deberán abrirse únicamente cuando sea necesario y cerrarse una vez que la operación haya sido completada, con el fin de evitar el consumo innecesario de recursos en el sistema.

Cualquier error relacionado con la base de datos deberán manejarse mediante excepciones SQL, registrando el error mediante el sistema de logging del programa y reempaquetandolo en la excepción de dominio correspondiente

Uso obligatorio de try-with-resources

Todas las conexiones, PreparedStatement y ResultSet deberán declararse en una sentencia try-with-resources para garantizar que se cierre automáticamente al finalizar el bloque

Con estándar
<pre>try(Connection connection = connectionManager.getConnection(); PreparedStatement prepareStatement = databaseConnection.prepareStatement(userQuery)) { preparedStatement.setString(1,email); try(ResultSet resultSet = prepareStatement.executeQuery()) { if(resultSet.next()) { // logic } } } catch(SQLException e) { throw new AuthenticateException("Error al autenticar" + e.getMessage()); }</pre>
Sin estándar
<pre>Connection conn = null; PreparedStatement ps = null; try { conn = connectionManager.getConnection(); ps = conn.prepareStatement(query); //logic } finally { if(ps != null) ps.close(); if(conn != null) conn.close(); }</pre>

Manejo de transacciones

Cuando una operación de negocio realiza dos o más escrituras (INSERT, UPDATE) que deben tener éxito o fallar como una unidad, deberá ejecutarse dentro de una transacción explícita sobre una sola conexión compartida. Esto evita registros huérfanos: si la primera escritura se confirma de forma independiente y la segunda falla, la base de datos queda en un estado inconsistente.

Con estándar

```
@Override
public boolean registerSubject(Subject subject) throws
OperationException {
    boolean isRegistered = false;
    String subjectQuery = "INSERT INTO ExperienciaEducativa (NRC,
nombreExperiencia, carrera, "
                        + "idPeriodoEscolar, seccion) VALUES (?, ?, ?, ?, ?);";
    String professorSubjectQuery = "INSERT INTO
Profesor_Imparte_Experiencia (NRC, numeroPersonal) "
                        + "VALUES (?, ?);";

    try (Connection databaseConnection =
connectionManager.getConnection()) {
        databaseConnection.setAutoCommit(false);

        try (PreparedStatement subjectStatement =
databaseConnection.prepareStatement(subjectQuery);
PreparedStatement professorStatement =
databaseConnection.prepareStatement(professorSubjectQuery)) {

            int subjectRows = subjectStatement.executeUpdate();
            int professorRows = professorStatement.executeUpdate();

            boolean bothInserted = subjectRows != NO_ROWS_AFFECTED
&& professorRows != NO_ROWS_AFFECTED;

            if (bothInserted) {
                databaseConnection.commit();
                isRegistered = true;
                LOGGER.log(Level.INFO, "Experiencia Educativa registrada
con éxito.");
            } else {
                databaseConnection.rollback();
                LOGGER.log(Level.WARNING, "No se afectaron filas; se revirtió
la operación.");
            }
        }
    }
}
```

```

    } catch (SQLException e) {
        LOGGER.log(Level.SEVERE, "Error de conexión al registrar la
Experiencia Educativa", e);
        throw new OperationException("No se pudo registrar la Experiencia
Educativa. "
            + "Intentelo más tarde", e);
    }
    return isRegistered;
}

```

Sin estándar

```

PreparedStatement          subjectStatement          =
databaseConnection.prepareStatement(subjectQuery);
subjectStatement.executeUpdate();

PreparedStatement          professorStatement        =
databaseConnection.prepareStatement(professorQuery);
professorStatement.executeUpdate();

```

Uso de PreparedStatement

Las consultas de SQL que reciba valores provenientes del usuario o de variables debe ejecutarse mediante PreparedStatement utilizando parámetros (?). No se puede concatenar valores directamente dentro de la cadena SQL, ya que esto expone al sistema a ataques de inyección SQL.

Con estándar

```

String userQuery = "SELECT idUsuario FROM Usuario WHERE email =
?"; try(PreparedStatement preparedStatement =
    databaseConnection.prepareStatement(userQuery)) {
    preparedStatement.setString(1,email);
}

```

Sin estándar

```

String userQuery = "SELECT idUsuario FROM Usuario WHERE email =
    " + email + " ";
Statement statement = databaseConnection.createStatement();
ResultSet rs = statement.executeQuery(userQuery);

```

Pruebas unitarias

Se utilizan JUnit 5(org.junit.jupiter) como framework de pruebas unitarias y la biblioteca Mockito para simular objetos. Toda clase DAO y clases con lógica deberá contar consus respectivos casos de prueba.

Ubicación y nomenclatura

- Las pruebas tendrán espacio dentro de un paquete “test”.
- El nombre de la clase de prueba será el nombre de la clase a probar más el sufijo Test.

Estructura de un Test

Cada método de prueba deberá seguir el patrón Arrange-Act-Assert, separado por líneas en blanco. La configuración común entre tests irá en un método anotado con @BeforeEach. Dentro de este método se deberá inicializar Mockito mediante MockitoAnnotations.openMocks(this) como primera instrucción, para activar las anotaciones @Mock declaradas en la clase.

Cada método de test solo debe contar con un assert, no debe tener 2 o más.

Con estándar

```
class UserDAOTest {  
  
    @Mock private MySQLConnectionManager connectionManager;  
    @Mock private Connection databaseConnection;  
    @Mock private PreparedStatement preparedStatement;  
    @Mock private ResultSet resultSet;  
  
    @BeforeEach
```

```

void          setUp()          throws          Exception          {
MockitoAnnotations.openMocks(this);

    userDao = new UserDao();
    Field field = UserDao.class.getDeclaredField("connectionManager");
field.setAccessible(true);
field.set(userDao, connectionManager);
when(connectionManager.getConnection()).thenReturn
(databaseConnection);
}

@Test
void registerUser_successful_returnsGeneratedId() throws Exception {
when(databaseConnection.prepareStatement(anyString(),    anyInt()))
.thenReturn(preparedStatement);
when(preparedStatement.executeUpdate()).thenReturn(1);
when(preparedStatement.getGeneratedKeys()).thenReturn(resultSet);
when(resultSet.next()).thenReturn(true);
when(resultSet.getInt(1)).thenReturn(42);

assertEquals(42, userDao.registerUser(
builderUser("Juan", "Pérez", "password12", 1)));
}

```

Sin estándar

```

class UserDAOTest {

    private MySQLConnectionManager connectionManager;
    private Connection databaseConnection;
    private PreparedStatement preparedStatement;
    private ResultSet resultSet;

    void setUp() throws Exception {
        userDAO = new UserDAO;
        Field field = UserDAO.class.getDeclaredField("connectionManager");
        field.setAccessible(true);
        field.set(userDAO, connectionManager);
        when(connectionManager.getConnection()).thenReturn
        (databaseConnection);
    }

    @Test
    void registerUser_successful_returnsGeneratedId() throws Exception {
        when(databaseConnection.prepareStatement(anyString(), anyInt()))
        .thenReturn(preparedStatement);
        when(preparedStatement.executeUpdate()).thenReturn(1);
        when(preparedStatement.getGeneratedKeys()).thenReturn(resultSet);
        when(resultSet.next()).thenReturn(true);
        when(resultSet.getInt(1)).thenReturn(42);

        assertEquals(42, userDAO.registerUser(
        builderUser("Juan", "Pérez", "password12", 1)));
    }
}

```

Métodos builders en pruebas

Cuando un test requiera construir un objeto complejo con muchos campos, se recomienda extraer la construcción a un método privado dentro de la propia clase de prueba con el prefijo builder (builderUser).

Validación de entrada y seguridad

Centralización de validaciones

Todas las validaciones de formularios deberán ubicarse en la clase utilitaria InputValidator. Los métodos de InputValidator deben devolver Optional<String>, vacío si el dato es válido o conteniendo un mensaje de error si no lo es. El nombre del campo se recibe como segundo parámetro para construir mensajes de error contextualizados.

Almacenamiento de contraseñas

Las contraseñas nunca se deben almacenar o transmitir en texto plano. Se utilizará el algoritmo Argon2id a través de la clase PasswordHasher para generar el hash al momento del registro y para verificar el hash almacenado al momento de la autenticación.

Con estándar
<pre>public boolean registerUser(User user) throws OperationException { //database logic String hashedPassword = PasswordHasher.hashPassword(user.getPassword()); user.setPassword(hashedPassword); }</pre>
Sin estándar
<pre>public boolean registerUser(User user) throws OperationException { //database logic user.setPassword(user.getPassword()); }</pre>

Gestión de sesión

La información del usuario autenticado en la sesión activa se gestionará mediante la clase SessionManager, la instancia del usuario deberá obtenerse mediante getInstance() y se proveerán medios para limpiar la sesión (clearSession)

DTOs y modelos de dominio

Estructura de un DTO

Todos los DTO deben contar con sus atributos privados, así como un constructor sin parámetros y uno con todos los parámetros, igualmente deberán tener sus respectivos getters y setters junto a la sobreescritura del método equals() si es necesario

Si una clase sobrescribe equals(Object), deberá sobrescribir también hashCode(), y ambos deben calcularse a partir de los mismos campos.

Con estándar

```

public class Subject{
    private String nrc;
    private String name;

    public Subject() {
    }

    public Subject(String nrc, String name) {
        this.nrc = nrc;
        this.name = name;
    }

    public String getNrc() {
        return nrc;
    }

    public void setNrc(String nrc) {
        this.nrc = nrc;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
    @Override
    public boolean equals(Object object) {
        boolean isEqual = false;
        if (this == object) {
            isEqual = true;
        } else if (object != null && getClass() == object.getClass()) {
            Subject other = (Subject) object;
            isEqual = nrc == other.nrc
                && Objects.equals(name, other.name);
        }
        return isEqual;
    }

    @Override
    public int hashCode() {
        return Objects.hash(nrc, name);
    }
}

```

Sin estándar

```
public class Subject{
    private String nrc;

    public Subject() {
    }

    public void setNrc(String nrc) {
        this.nrc = nrc;
    }
}
```

Manejo de documentos

Manejo y Guardado de Archivos

El procesamiento y almacenamiento de los archivos adjuntos al expediente se centralizará a través de la clase utilitaria FileManager. Este componente será el responsable de gestionar el guardado físico de los documentos en el directorio del sistema, asegurando que las rutas de acceso se registren y vinculen correctamente para su posterior consulta.

Para garantizar la seguridad, estabilidad e integridad de la aplicación, toda carga de archivos deberá pasar previamente por la clase FileValidator.

Generación Automática de Documentos

La expedición de los formatos oficiales (Reporte Parcial, Reporte Mensual, Reporte Final y Autoevaluación) estará automatizada utilizando JasperReports.

El flujo de generación se basará en el uso de plantillas visuales estandarizadas. Posteriormente, la capa lógica del programa cargará estas plantillas y, mediante el mapeo de parámetros, inyectará dinámicamente la información correspondiente al documento. Finalmente, el código procesa estos datos para exportar el documento resultante, dejándolo listo para su guardado o visualización en formato PDF.

Gestión de ventanas en JavaFX

Navegación entre vistas

La navegación entre vistas deberá centralizarse en métodos protegidos de la clase WindowHandler, se invocará el método navigateTO(String fxmlPath) heredado,

evitando el duplicado de las instancias FXMLLoader, Scene y Stage.

Con estándar

```
protected void navigateTo(String fxml) {
    try {

        FXMLLoader loader = new FXMLLoader(getClass().getResource(fxml));
        Parent root = loader.load();
        Stage stage = new Stage();
        Stage.setScene(new Scene(root));
        stage.show();
    } catch (IOException e) {
        showError("Error al cargar la pantalla",null);
    }
}

navigateTo("/uv/lis/GUI/view/FXMLConsultStudent.fxml");
```

Sin estándar

```
@FXML
public void goToConsultStudent(ActionEvent event) {
    try {
        FXMLLoader loader = new FXMLLoader(getClass().getResource(
            "/uv/lis/GUI/view/FXMLConsultStudent.fxml"));
        Parent root = loader.load();
        Stage stage = new Stage();
        stage.setScene(new Scene(root));
        stage.show();
    } catch (IOException e) {
        LOGGER.log(LEVEL.WARNING, "Error al cambiar de ventana");
    }
}
```

Mensajes al usuario

Los mensajes al usuario se mostrarán mediante dos mecanismos:

- Un label declarado en la propia vista(verde para éxito, rojo para error), gestionado por los métodos showError y showSuccess del ValidationHandler.

- Los mensajes al usuario serán en español. Los mensajes técnicos para el LOGGER también serán en español para mantener la consistencia entre el equipo.

Dependencias externas y seguridad

Para la realización de ciertos casos de uso en el sistema se utilizaron librerías externas para la implementación de sus funcionalidades, por ejemplo:

- MySQL-Connector.-Para establecer la conexión entre el sistema y la base de datos MySQL mediante el protocolo JDBC.
- JavaFX-Controls y JavaFX-FXML.-Para la construcción de la interfaz gráfica de usuario y la vinculación de las vistas FXML con sus controladores.
- Argon2-jvm.-Para el hashing y verificación de contraseñas de los usuarios mediante el algoritmo Argon2id.
- JasperReports.-Para conectar las interfaces a una plantilla y que la información ingresada se rellene directamente en el reporte generado. Dentro de esta librería se usa las clases JasperPrint y JRBasePrintPage para la elaboración de objetos de prueba simulados en los Tests que generan documentos.
- iText.-Para la exportación de los reportes generados por JasperReports en formato PDF.
- Jakarta.mail-api y Angus-mail.-Para el envío de correos electrónicos desde el sistema hacia los usuarios.
- JUnit Jupiter.-Para la realización de pruebas unitarias a las clases DAO y la lógica del sistema.
- Mockito-Core y Mockito-JUnit-Jupiter.-Para la simulación de objetos y dependencias durante la ejecución de las pruebas unitarias.
- TestFX (testfx-core y testfx-junit5).-Para la realización de pruebas automatizadas sobre la interfaz gráfica construida con JavaFX.

Vulnerabilidades conocidas

iText 2.1.7

Presenta una vulnerabilidad (CVE-2017-9096) en el procesamiento de documentos XML/PDF que permite, mediante un documento manipulado, leer archivos del sistema o realizar solicitudes internas al servidor (inyección de entidades externas XML, XXE/SSRF). Se declara de forma directa en el pom.xml y, además, es utilizada de forma transitiva por JasperReports.

Por qué no afecta al sistema: la aplicación genera PDFs únicamente a partir de plantillas predefinidas y controladas por el equipo, y no procesa documentos ni XML subidos por los usuarios, por lo que el documento de entrada malicioso que requiere esta vulnerabilidad no es alcanzable.

JasperReports 6.21.3

Esta dependencia presenta dos riesgos. El primero es indirecto: arrastra iText 2.1.7 como dependencia, por lo que hereda el riesgo descrito anteriormente. El segundo es una vulnerabilidad crítica de deserialización Java insegura (CVE-2025-10492) que afecta a todas las versiones hasta la 7.0.3 inclusive; por lo tanto, la versión 6.21.3 utilizada se encuentra dentro del rango afectado. Su explotación requiere deserializar datos provenientes de una fuente externa, lo que podría derivar en ejecución remota de código.

Por qué no afecta al sistema: el riesgo heredado de iText se reduce al ya descrito; y respecto a la deserialización, el sistema no deserializa objetos de JasperReports provenientes de fuentes externas, pues las plantillas se compilan en tiempo de ejecución desde archivos .jrxml controlados por el proyecto y el llenado se realiza con parámetros generados internamente.

Angus-mail 2.0.4

Existe una vulnerabilidad (CVE-2025-7962) que permite a un atacante introducir saltos de línea dentro de alguno de esos campos para añadir cabeceras adicionales o inyectar comandos en el servidor de correo, logrando que los mensajes se envíen a destinatarios no previstos. Esta vulnerabilidad afecta a las versiones anteriores a la 2.0.4.

Por qué no afecta al sistema: los datos utilizados para construir los correos provienen del sistema mismo y no de entradas directas del usuario sin validar. Además, la versión utilizada (2.0.4) corrige esta vulnerabilidad internamente.

MySQL-connector-j 8.3.0

Existe una vulnerabilidad (CVE-2023-22102) que afecta a la versión 8.1.0 y anteriores, la cual permite a un atacante con acceso a la red comprometer el conector y, a través de él, el servidor MySQL.

Por qué no afecta al sistema: la versión 8.3.0 utilizada en el proyecto incorpora la corrección correspondiente y queda fuera del rango afectado.

Argon2-jvm 2.11

No se registran vulnerabilidades (CVEs) directas conocidas para esta librería en la versión utilizada; corresponde a un envoltorio sobre la implementación de referencia de Argon2.

JavaFX 21.0.2 (Controls y FXML)

Las vulnerabilidades históricas asociadas al componente JavaFX se reducen a despliegues en entornos de tipo sandbox (Java Web Start o applets que cargan código no confiable desde Internet).

Por qué no afecta al sistema: la aplicación es de escritorio y no carga ni ejecuta código no confiable de terceros, por lo que ese modelo de amenaza no se aplica.

Librerías de prueba: JUnit Jupiter 5.9.3, Mockito 5.11.0 y TestFX 4.0.18

No presentan vulnerabilidades conocidas relevantes en las versiones utilizadas.